Recall@k, Worked Out by Hand

The one number that tells you whether your retriever — or your approximate index — is actually finding what it should: the fraction of all relevant items that appear in the top k. Computed by hand in both senses: ANN recall and retrieval recall.

What this document does. It defines recall@k, computes it for an ANN index (did the approximate search find the true nearest neighbours?) and for a retriever (did the ranked list surface the relevant documents?), shows how recall behaves as k grows, and contrasts it with precision so you know which to optimise. Numbers from Appendix A.

Why it lives in Track 1.1: it is the honest scoreboard for the speed/accuracy trades made by HNSW (#9) and quantization (#10, #11).

1 The formula

Recall@k is the share of the relevant set that made it into the top k results:

$$\text{Recall@}k = \frac{|\,\{\text{relevant items}\} \cap \{\text{top-}k\text{ retrieved}\}\,|}{|\,\{\text{relevant items}\}\,|}$$

The denominator is *all* the relevant items that exist (whether or not you found them); the numerator is how many of them you actually returned in the first k. It ranges $[0, 1]$; 1.0 means you missed nothing.

> **Intuition**
>
> Recall answers "*of everything I should have found, how much did I?*" It is the metric for *completeness*, and it is the right one whenever a miss is expensive: RAG (a missing chunk the LLM never sees can't be reasoned over), legal discovery, medical retrieval. Its blind spot is junk — recall says nothing about how much irrelevant material you also returned. That is precision's job, and the two always trade against each other.

2 Sense 1 — ANN recall (did approximate search find the truth?)

For a vector index, "relevant" means *the true (exact) nearest neighbours*. The exact top-5 for a query are the ids $\{1, 2, 3, 4, 5\}$. The approximate index (HNSW + PQ) returns, in order:

$$[\,2,\ 7,\ 1,\ 3,\ 9,\ 5,\ 8,\ 4,\ 10,\ 11\,].$$

Recall@5. Top-5 returned $= \{2, 7, 1, 3, 9\}$; intersect with the true $\{1, 2, 3, 4, 5\}$ gives $\{1, 2, 3\}$:

$$\text{Recall@5} = \frac{3}{5} = 0.60.$$

The approximate search missed true neighbours 4 and 5 in its top-5 — the price of sub-linear speed.

Recall@10. Widen to the top-10; now $\{2, 7, 1, 3, 9, 5, 8, 4, 10, 11\}$ contains all of $\{1, 2, 3, 4, 5\}$:

$$\text{Recall@10} = \frac{5}{5} = 1.00.$$

The neighbours were found — just ranked a little late. This is the classic ANN tuning story: raise `efSearch` (or k) until recall@k clears your bar.

> **Mini-example · recall is monotonic in k**
>
> Recall@k can only *rise* (or stay flat) as k grows — a larger window can only add hits, never remove them. Here it climbed $0.60 \to 1.00$ between k=5 and k=10. So "recall@k" is meaningless without its k: `recall@1` and `recall@100` measure very different promises. Always report the k.

3 Sense 2 — retrieval recall (did the retriever surface the right docs?)

Same formula, "relevant" now means human-judged relevant documents. Suppose three documents are relevant to a query, $\{d_1, d_3, d_7\}$, and the retriever's ranked list is

$$[\, d_2,\ d_1,\ d_5,\ d_3,\ d_9,\ d_7,\ d_4 \,].$$

$$\text{Recall@3}: \ \{d_2, d_1, d_5\} \cap \{d_1, d_3, d_7\} = \{d_1\} \ \Rightarrow\ \tfrac{1}{3} = 0.333,$$

$$\text{Recall@5}: \ \{d_2, d_1, d_5, d_3, d_9\} \cap \{d_1, d_3, d_7\} = \{d_1, d_3\} \ \Rightarrow\ \tfrac{2}{3} = 0.667.$$

Only at $k = 6$ (where d_7 appears) does recall reach 1.0. If your RAG pipeline only feeds the top-3 chunks to the LLM, recall@3 = 0.33 means it is reasoning with a third of the evidence — a retrieval problem masquerading as a generation problem.

4 Recall vs precision — pick by failure cost

- **Recall@k** $= \dfrac{\text{relevant found}}{\text{relevant total}}$ — penalises *misses*. Optimise when leaving something out is the costly error.
- **Precision@k** $= \dfrac{\text{relevant found}}{k}$ — penalises *junk*. Optimise when a noisy top-k is the costly error.
- **The trade:** increasing k raises recall but usually lowers precision (more results, more noise). The right operating point depends on what the downstream consumer (an LLM, a human, a ranker) does with the list.

In RAG the usual move is *retrieve* at high k for recall, then *re-rank* (Track 3.3) to restore precision in the final few — get everything, then sort it.

Equation cheat-sheet

$$\text{Recall@}k = \frac{|\text{relevant} \cap \text{top-}k|}{|\text{relevant}|}, \qquad \text{Precision@}k = \frac{|\text{relevant} \cap \text{top-}k|}{k}$$

Recall@k is non-decreasing in k, ANN: "relevant" = exact nearest neighbours

Appendix A · Reference implementation

```
def recall_at_k(retrieved, relevant, k):
    hits = set(retrieved[:k]) & set(relevant)
    return len(hits) / len(relevant)

true_nn = {1,2,3,4,5}
ann     = [2,7,1,3,9,5,8,4,10,11]
recall_at_k(ann, true_nn, 5)    # 0.60
recall_at_k(ann, true_nn, 10)   # 1.00

relevant = {"d1","d3","d7"}
ranked   = ["d2","d1","d5","d3","d9","d7","d4"]
recall_at_k(ranked, relevant, 3)   # 0.333
recall_at_k(ranked, relevant, 5)   # 0.667
```

Internal learning material. Numbers reproducible from the appendix code. v1.0